

AI-Driven Multi-Objective Orchestration and Adaptive Scheduling for Noise-Aware Distributed Quantum Computing

Author Name

Department / Institution

Affiliation

City, Country

email@domain

Abstract—Distributed quantum computing (DQC) couples several noisy intermediate-scale quantum (NISQ) processors so that circuits exceeding the capacity or fidelity of any single device can be executed cooperatively. Operating such a fleet requires an orchestration layer that decides which processor(s) run which circuit, in what order, subject to objectives that are simultaneously in tension: delivered fidelity, decoherence-bounded deadlines, throughput, and load balance. Classical schedulers optimize none of the quantum-specific objectives, and a naive fidelity-greedy policy ignores deadlines and collapses under contention. We present AMOS, an AI-driven, multi-objective orchestrator with an adaptive, noise-aware scheduling core. AMOS (i) formulates placement as the maximization of a weighted multi-objective host score that *satisfices* fidelity—rewarding tasks that clear a fidelity threshold rather than maximizing fidelity unconditionally—while jointly optimizing deadline feasibility, speed, and load; (ii) orders tasks by least laxity to protect coherence budgets under overload; and (iii) adapts online to calibration drift through a lightweight reliability estimator that derates processors whose realized fidelity falls below their datasheet. We evaluate AMOS in a discrete-event simulator of eight heterogeneous QPUs with decoherence, entanglement-mediated circuit distribution, and calibration drift. AMOS matches the best baseline when the fleet is uncontended and improves goodput by up to $\sim 2\times$ under heavy overload while sustaining a $>99\%$ deadline-hit rate. A weight-sensitivity study shows the delivered-fidelity/goodput operating point is robust across the full *satisfice-to-maximize* range (variation < 1 pp), which justifies the *satisficing* default. An ablation shows the structured multi-objective policy, not the learner, drives the gain in stationary conditions; online adaptation earns its place only under non-stationarity, where it recovers a consistent fraction of the goodput lost after a device-degradation event. We report these findings—including the limits of the learning component—transparently and release the simulator for reproduction.

Index Terms—distributed quantum computing, quantum orchestration, multi-objective scheduling, NISQ, decoherence, entanglement distribution, online adaptation, noise-aware scheduling

I. INTRODUCTION

Individual quantum processors are limited in qubit count and, more fundamentally, in the fidelity and coherence time available to a single device [2]. Distributed quantum computing (DQC) mitigates these limits by interconnecting multiple processors—within a chassis, a data center, or an eventual

quantum network—so that a large circuit can be partitioned and executed across nodes using entanglement and classical communication [1], [3]; distributed execution has recently been demonstrated across an optical network link [12]. As quantum cloud providers move toward multi-tenant operation over pools of heterogeneous devices [8], they need an *orchestration layer*: a control plane that admits a stream of quantum tasks (circuits) and maps them onto a heterogeneous QPU fleet over time.

Orchestrating a quantum fleet is not classical job scheduling with new labels. Four properties make off-the-shelf policies inadequate:

- **Fidelity is a first-class outcome.** Without error correction, a NISQ execution returns a result whose usefulness is governed by process fidelity; two placements that both “complete” can differ from useful to worthless.
- **Deadlines are physical.** A circuit must finish within a coherence-limited budget; gate error and decoherence jointly cap the useful depth on a given device.
- **Distribution is not free.** Executing across k nodes introduces non-local gates realized by entanglement (gate teleportation), each consuming an EPR pair and adding latency and a multiplicative fidelity penalty [4], [5].
- **Devices are non-stationary.** Calibration drifts; a processor that looks best on its datasheet can be transiently or persistently unreliable [9].

These properties place the orchestrator in a genuinely *multi-objective* regime—delivered fidelity, deadline satisfaction, throughput, and load balance must be traded against one another—and in a *non-stationary* one, where the datasheet the orchestrator relies on may be stale. Classical throughput/load-oriented schedulers ignore fidelity, while a fidelity-greedy policy ignores deadlines and contention and, as we show, collapses under load.

Contributions.

- 1) We formulate noise-aware DQC orchestration over heterogeneous NISQ processors as multi-objective placement plus deadline-aware ordering, with a *goodput* objective coupling a decoherence-bounded deadline and a

fidelity threshold (Section III).

- 2) We propose **AMOS**, an AI-driven orchestrator whose scheduling core combines least-laxity ordering, a *satisficing* multi-objective host score, and an online reliability estimator that adapts to calibration drift (Section IV).
- 3) We build a discrete-event simulator modeling decoherence, entanglement-mediated distribution, and per-device drift, and evaluate AMOS against four baselines with a controlled ablation (Sections V–VI).
- 4) We report results honestly: AMOS improves goodput by up to $\sim 2\times$ under overload; its operating point is robust to the fidelity-weighting knob; and online learning is negligible under stationarity but consistently helpful under non-stationarity. We discuss the model’s limitations explicitly (Section VII).

II. RELATED WORK

DQC foundations, surveys, and hardware. The DQC landscape is mapped by recent surveys spanning single-QPU limits to high-performance quantum computing [1], [2], reviews of recent progress [14], and taxonomies of the path toward fault-tolerant DQC [13]. The networked substrate is framed by the quantum-internet vision and its building blocks [3], [15]. Crucially, DQC is no longer only theoretical: recent hardware demonstrations couple processors with real-time classical communication [6] and even execute a distributed algorithm across an optical network link [12], making the control-plane questions we study concrete.

Circuit cutting, knitting, and multi-core mapping. Distribution is made concrete by circuit cutting/knitting, which decomposes a large circuit into subcircuits at a classical/entanglement overhead that grows with the number of cuts [4], [5]. Subsequent work tightens this overhead for wire cuts [16], multiple two-qubit unitaries [17], and clustered simulation [18]. On the compiler side, modular compilation frameworks [19], qubit-assignment heuristics for multi-core architectures [20], and learned circuit partitioning [21] target the *intra-job* mapping. We adopt an abstracted form of the resulting distribution cost in our execution model and treat cutting/mapping as orthogonal to the *across-job*, *over-time* orchestration problem.

Quantum cloud scheduling and orchestration. Closest to our work, several recent efforts schedule quantum jobs across a fleet. Bahrani et al. [7] study resource management and circuit scheduling for DQC interconnect networks, Chandra et al. [11] schedule the underlying network operations, Ferrari et al. [22] address execution management of DQC jobs, and Du et al. [5] co-optimize circuit cutting and scheduling with dynamic EPR pairs. Learning-based orchestrators are increasingly common: DRLQ places tasks with a deep Q-network to cut completion time [8], QFOR adds fidelity awareness to a deep reinforcement-learning orchestrator [9], and Luo et al. apply policy-gradient RL to adaptive job scheduling in quantum clouds [10]. Reviews and simulation toolkits situate these efforts [23], [24].

Positioning. AMOS differs from the above in two respects. First, its scheduling core is a deliberately lightweight, interpretable multi-objective *satisficing* policy with an online reliability derate—no deep network, no reward shaping—so its behavior is auditable and its weights map directly onto operator objectives. Second, we report the value of the learned component *honestly*: an ablation shows it is negligible in stationary conditions and helps only, and modestly, under non-stationarity—a distinction the deep-RL orchestrators above do not isolate. To our knowledge, we are the first to couple a decoherence-bounded deadline and a fidelity threshold into a single goodput objective for across-fleet orchestration and to quantify when adaptation actually pays.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. Processors

The fleet is a set $\mathcal{Q} = \{q_1, \dots, q_M\}$ of heterogeneous QPUs. Each q_j has qubit capacity n_j , nominal per-gate error ε_j , per-layer time τ_j , coherence time T_j (a T_2 -like budget), and a calibration-instability factor δ_j . The datasheet exposes $(n_j, \varepsilon_j, \tau_j, T_j)$ to the orchestrator; δ_j and moment-to-moment fluctuations are *not* observable a priori.

B. Tasks

Tasks arrive as a stream. Task i has arrival a_i , width w_i , depth L_i , gate count g_i , and absolute deadline d_i . Arrivals are Poisson with rate set by an offered-load parameter ρ relative to aggregate fleet capability, so larger ρ means more contention. Deadlines are $d_i = a_i + \sigma_i \hat{t}_i$ for a nominal estimate $\hat{t}_i$ and laxity $\sigma_i \in [2, 6]$, modeling coherence-bounded urgency.

C. Execution and fidelity

For a monolithic placement ($w_i \leq n_j$) on q_j , execution time is $t_{ij} = L_i \tau_j$ and the estimated process fidelity is

$$F_{ij} = \underbrace{(1 - \varepsilon_j)^{g_i}}_{\text{gate errors}} \cdot \underbrace{\exp(-t_{ij}/T_j)}_{\text{decoherence}}. \quad (1)$$

When w_i exceeds every free single device, the task is distributed over a set S of $k = |S|$ processors, introducing $c_i = \lceil \beta L_i (k - 1) \rceil$ non-local gates; each consumes an EPR pair of link fidelity F_{link} and adds latency t_{epi} :

$$t_i^{\text{dist}} = L_i \max_{j \in S} \tau_j + c_i t_{\text{epi}}, \quad (2)$$

$$F_i^{\text{dist}} = \left(\prod_{j \in S} (1 - \varepsilon_j)^{g_i/k} \right) e^{-t_i^{\text{dist}} / \min_{j \in S} T_j} F_{\text{link}}^{c_i}. \quad (3)$$

Equations (1)–(3) are intentionally first-order models (Section VII); they capture the qualitative couplings—error accumulation with gate count, decoherence with wall-clock time, and a compounding entanglement penalty with the degree of distribution.

D. Calibration drift

The *realized* outcome differs from the datasheet. At execution, ε_j and $1/T_j$ are each perturbed by an independent lognormal factor with log-standard-deviation $\sigma_{\text{drift}}\delta_j$, so unstable devices ($\delta_j > 1$) fluctuate more. A degradation event may persistently inflate a device's error rate mid-stream (Section V) *without* updating the datasheet the orchestrator sees.

E. Multi-objective goodput objective

For each dispatched task we define objective components at placement: a fidelity margin $m_i = F_i - F_{\text{th}}$, a deadline slack $\ell_i = d_i - f_i$ (with realized finish f_i), execution speed, and incremental fleet load. A task *succeeds* iff it clears both the deadline and the fidelity threshold:

$$\text{success}_i = \mathbb{I}[f_i \leq d_i] \cdot \mathbb{I}[F_i \geq F_{\text{th}}]. \quad (4)$$

The orchestrator maximizes *goodput* $G = \frac{1}{N} \sum_i \text{success}_i$, the scalar that couples the fidelity and deadline objectives; delivered fidelity, deadline-hit rate, and utilization are reported as the constituent objectives. Even the offline, static-fidelity version of this assignment is NP-hard (it generalizes deadline-constrained unrelated-machine scheduling), motivating an on-line heuristic.

IV. AMOS: ADAPTIVE MULTI-OBJECTIVE ORCHESTRATOR

AMOS runs at every orchestration event (arrival or completion). It maps a subset of pending tasks onto free processors through three mechanisms (Algorithm 1).

A. Least-laxity ordering

Pending tasks are considered in increasing laxity $\ell_i = d_i - t_{\text{now}} - \min_j t_{ij}$, an earliest-deadline-first discipline that protects tasks at risk of exhausting their coherence budget. This directly serves the deadline objective and is the main reason AMOS resists overload where fidelity-greedy (arrival-ordered) policies do not.

B. Satisficing multi-objective placement

For a task and candidate host-set h (a fitting single device, or one distributed set), AMOS scores

$$s(i, h) = w_F \Phi(\hat{F}_{ih} r_{ih}) + w_D \psi_{ih} + w_T \frac{t_i^{\min}}{t_{ih}} - w_L \bar{u}_h, \quad (5)$$

where $\hat{F}_{ih}$ is datasheet fidelity from (1)–(3), r_{ih} is a learned reliability derate (Section IV-C), $\psi_{ih} \in \{-1, +1\}$ flags deadline feasibility, $t_i^{\min}$ is the fastest feasible time, and $\bar{u}_h$ is an exponential-moving load estimate. The fidelity mapping Φ blends a satisficing and a maximizing view via an orchestration knob $\lambda \in [0, 1]$:

$$\Phi(F) = (1 - \lambda) \underbrace{\left(1 + e^{-\gamma(F - F_{\text{th}})}\right)^{-1}}_{\text{satisfice}} + \lambda \underbrace{F}_{\text{maximize}}. \quad (6)$$

At $\lambda=0$ (default) the fidelity term saturates once the threshold is safely cleared, so AMOS does not spend a scarce clean-but-slow device on a task that is already safe elsewhere;

Algorithm 1 AMOS: one orchestration event

```

1: Input: time  $t$ , pending  $\mathcal{P}$ , free hosts  $\mathcal{F}$ 
2: sort  $\mathcal{P}$  by ascending laxity  $\ell_i$ 
3: for each task  $i \in \mathcal{P}$  do
4:    $\mathcal{C} \leftarrow$  fitting single hosts in  $\mathcal{F}$ , else one distributed set
5:   if  $\mathcal{C} = \emptyset$  then continue
6:   end if
7:   for each  $h \in \mathcal{C}$  do
8:     score  $s(i, h)$  via (5)–(6) with derate  $r_{ih}$ 
9:   end for
10:   $h^* \leftarrow \arg \max_h s(i, h)$ 
11:  if  $t + t_{ih^*} > d_i$  then continue ▷ skip hopeless
12:  end if
13:  assign  $i \rightarrow h^*$ ; mark busy; update load EMA
14: end for
15: on completion of  $i$  on  $h$ : update rel, exp for  $(x_i, h)$ 

```

at $\lambda = 1$ it maximizes delivered fidelity like a fidelity-greedy policy. A host-set whose best option cannot meet the deadline is skipped rather than wasted. The weight vector (w_F, w_D, w_T, w_L) exposes the multi-objective trade-off to the operator; we use (1.6, 1.3, 0.7, 0.25) throughout and study sensitivity in Section VI.

C. Online reliability adaptation

Datasheet fidelity $\hat{F}$ is systematically optimistic for unstable devices. AMOS maintains, per discretized task-state x_i (width/depth buckets) and processor j , two exponential moving averages: realized success $\text{rel}(x, j)$ and datasheet-predicted success $\text{exp}(x, j)$. The derate is their smoothed ratio

$$r_{ij} = \min\left(1.1, \frac{\text{rel}(x_i, j) + \eta}{\text{exp}(x_i, j) + \eta}\right), \quad (7)$$

driving < 1 for devices that underperform their spec and $= 1$ for unseen pairs (default trust). This is a lightweight, interpretable alternative to a deep policy; it needs no reward shaping and adapts within tens of executions. Setting $r_{ij} \equiv 1$ gives the ablation **AMOS-S**.

Each event costs $O(|\mathcal{P}| |\mathcal{F}|)$ score evaluations, negligible relative to circuit execution for a modest fleet.

V. EXPERIMENTAL SETUP

Fleet. We model $M=8$ heterogeneous QPUs spanning the NISQ design space: capacities 12–65 qubits, per-gate errors 6×10^{-4} to 9×10^{-3} , layer times 3.0 – 8.0×10^{-7} s, and T_2 from 7.0×10^{-5} to 3.0×10^{-4} s. Two nominally attractive devices are made *calibration-unstable* ($\delta_j=2.4$ – 2.6) and one clean device is made *stable* ($\delta_j=0.6$), so datasheet ranking and realized reliability diverge. Fixed physics: $F_{\text{th}}=0.50$, $F_{\text{link}}=0.94$, $t_{\text{ep}}=2 \mu\text{s}$, $\beta=0.12$, $\sigma_{\text{drift}}=0.35$.

Workload. Widths are mostly ≤ 12 with medium/large tails; $\sim 12\%$ of tasks exceed the largest free single device and require distribution. Depths are 6–55 layers, $g_i \approx 0.5 w_i L_i$. The feasibility ceiling (fraction runnable above threshold on the best device) is ≈ 0.70 , and placement is decisive: $\approx 69\%$ feasible on the cleaner half of the fleet versus $\approx 47\%$ on the

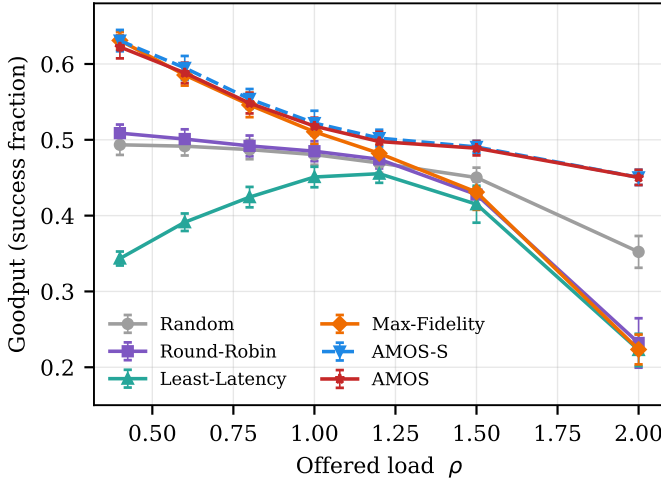


Fig. 1. Goodput vs. offered load ρ (mean $\pm$ s.d., 15 seeds). AMOS and its static ablation track the best baseline when uncontended and degrade gracefully under overload, where fidelity- and latency-greedy policies collapse.

noisier half. This is a deliberately non-degenerate regime in which orchestration can matter (Section VII).

Baselines. *Random*; *Round-Robin*; *Least-Latency* (FCFS, fastest free device); *Max-Fidelity* (FCFS, highest datasheet fidelity). We also report **AMOS-S** (no learning).

Protocol. Each configuration uses $N=1500$ tasks and 15 seeds; all policies see the *same* workload and *same* drift realization per seed (paired comparison). We sweep offered load $\rho \in \{0.4, \dots, 2.0\}$ and report mean and s.d. of goodput, deadline-hit rate, mean delivered fidelity, and utilization. The full simulator and orchestrator code accompany the paper; all runs are seeded, so every figure and table reproduces exactly. Concretely, `final_run.py` generates Figs. 1, 2, and 4 with Tables I and III; `pareto_run.py` with `make_sens_fig.py` generate Fig. 3 and Table II.

VI. RESULTS

A. Goodput vs. offered load

Figure 1 and Table I summarize the sweep. When uncontended ($\rho=0.4$) deadlines rarely bind and fidelity-greedy placement is already near the feasibility ceiling: Max-Fidelity and AMOS-S tie at 0.63 goodput, AMOS within noise. AMOS earns its advantage as contention rises. At $\rho=1.5$, AMOS-S reaches 0.491 versus 0.431 for Max-Fidelity (+13.9% relative, +6.0 pp) and 0.450 for Random. At heavy overload ($\rho=2.0$) the separation is stark: Max-Fidelity, Least-Latency, and Round-Robin all collapse to ≈ 0.22 – 0.23 as they miss deadlines and thrash slow devices, whereas AMOS sustains 0.450—roughly a $2\times$ goodput improvement over fidelity-greedy and +28% over the best baseline.

B. Where the gain comes from

Table I isolates the mechanism. Under load, AMOS holds a deadline-hit rate of ≈ 0.995 – 0.996 while baselines fall to 0.937–0.967 at $\rho=1.5$: least-laxity ordering does most of the

TABLE I
OBJECTIVE COMPONENTS AT $\rho=1.0$ AND $\rho=1.5$ (MEAN, 15 SEEDS).
G=GOODPUT, DL=DEADLINE-HIT, F=MEAN FIDELITY, U=UTILIZATION.

Offered load $\rho = 1.0$				
Policy	G	DL	F	U
Random	0.480	0.986	0.459	0.824
Round-Robin	0.485	0.986	0.460	0.826
Least-Latency	0.451	0.981	0.440	0.840
Max-Fidelity	0.511	0.988	0.478	0.826
AMOS-S	0.522	0.996	0.470	0.823
AMOS	0.518	0.996	0.468	0.822
Offered load $\rho = 1.5$				
Policy	G	DL	F	U
Random	0.450	0.967	0.475	0.982
Round-Robin	0.428	0.943	0.467	0.991
Least-Latency	0.415	0.937	0.466	0.994
Max-Fidelity	0.431	0.941	0.469	0.996
AMOS-S	0.491	0.995	0.474	0.991
AMOS	0.489	0.995	0.473	0.986

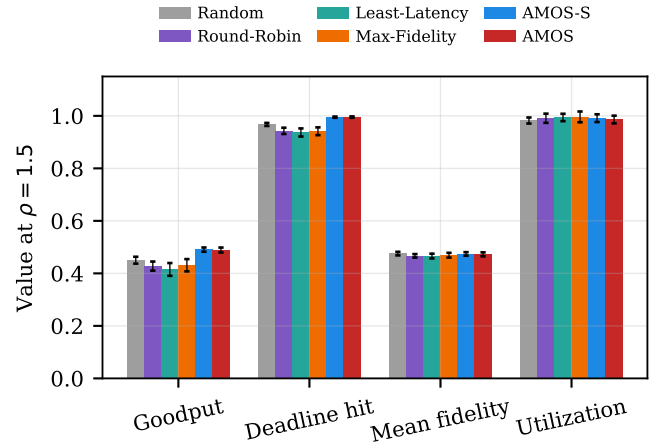


Fig. 2. Cross-objective comparison at $\rho=1.5$. AMOS raises goodput and deadline-hit rate while holding fidelity and utilization comparable—trading surplus fidelity for schedule feasibility.

work. Notably, AMOS’s mean delivered fidelity is slightly *lower* than Max-Fidelity’s at $\rho=1.0$ (0.468 vs. 0.478) even though its goodput is higher—the satisficing trade-off made explicit: AMOS accepts adequate (above-threshold) fidelity to meet more deadlines, and goodput, not raw fidelity, is the objective. Figure 2 shows the cross-objective picture at $\rho=1.5$.

C. Multi-objective operating point: weight sensitivity

The knob λ in (6) moves the fidelity objective from pure satisficing ($\lambda=0$) to pure maximization ($\lambda=1$). Figure 3 and Table II sweep it. The result is a *robustness* finding rather than a wide Pareto front: across the entire range, mean delivered fidelity rises by at most +0.006 and goodput changes by at most -0.005 (both within seed noise). In this regime delivered

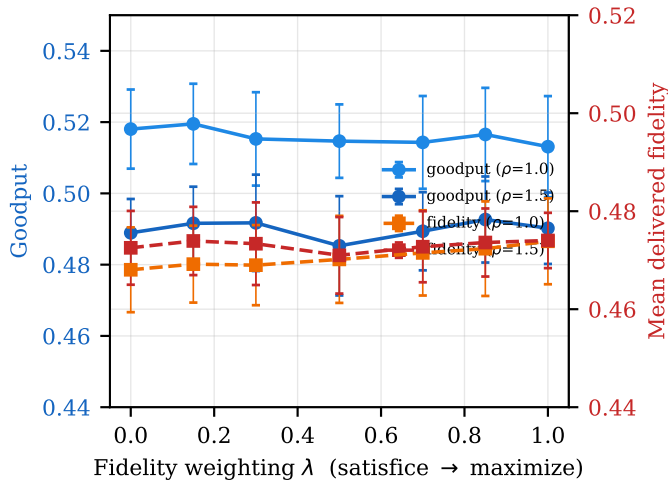


Fig. 3. Operating-point sensitivity to the fidelity-weighting knob λ (satisfice $\rightarrow$ maximize; mean $\pm$ s.d., 15 seeds). Both goodput (left axis) and mean delivered fidelity (right axis) are essentially flat, so the operating point is robust and the satisficing default is well-justified.

TABLE II
WEIGHT-SENSITIVITY OF AMOS TO THE FIDELITY KNOB λ (MEAN, 15 SEEDS). VARIATION IS WITHIN SEED NOISE.

λ	$\rho = 1.0$			$\rho = 1.5$		
	G	F	DL	G	F	DL
0.0	0.518	0.468	0.996	0.489	0.473	0.995
0.5	0.515	0.470	0.996	0.485	0.471	0.996
1.0	0.513	0.474	0.996	0.490	0.474	0.996

fidelity of successful tasks is pinned by device physics and the binding deadline constraint, so weighting fidelity more heavily buys almost no delivered fidelity while slightly reducing goodput at high load. This justifies the satisficing default ($\lambda=0$): it is at or near the best operating point, and the orchestrator’s behavior is not brittle to this weight. Where routing has more redundancy the knob would gain leverage; we note this as a scope condition (Section VII).

D. Ablation: what does online learning add?

Comparing AMOS and AMOS-S in Table I, the two are statistically indistinguishable across the stationary sweep (0.518 vs. 0.522 at $\rho=1.0$; 0.489 vs. 0.491 at $\rho=1.5$). We state this plainly: *when devices are stationary, the datasheet fidelity model already captures the signal and online reliability learning adds no meaningful benefit over the well-designed static policy*. The structured multi-objective placement, not the learner, drives the advantage over baselines.

E. Non-stationarity: when adaptation helps

The learner’s value appears when the datasheet goes stale. We repeat the $\rho=1.0$ experiment but, after 40% of the stream resolves, inflate the realized error rate of the two devices datasheet-greedy policies most prefer by $10\times$ (a persistent degradation event; the datasheet is *not* updated). Figure 4

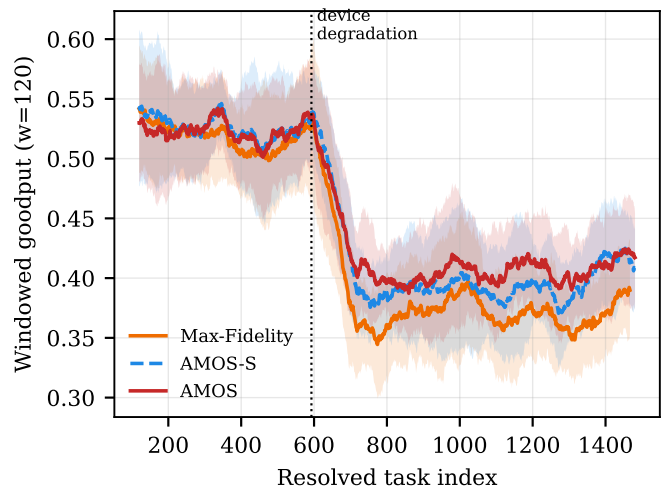


Fig. 4. Windowed goodput under a mid-stream device-degradation event (dotted line). Datasheet-greedy Max-Fidelity keeps routing to now-degraded devices; AMOS learns to derate them and recovers a larger share of goodput.

TABLE III
GOODPUT BEFORE/AFTER THE DEGRADATION EVENT ($\rho=1.0$, 15 SEEDS).

Policy	Pre	Post	Drop
Max-Fidelity	0.524	0.372	−0.152
AMOS-S	0.531	0.395	−0.136
AMOS	0.527	0.406	−0.122

plots windowed goodput and Table III reports pre/post means. All policies drop at the change point, but AMOS recovers the most: post-change goodput 0.406 versus 0.395 (AMOS-S) and 0.372 (Max-Fidelity), i.e. the online derate cuts the post-degradation loss from 0.152 and 0.136 to 0.122. The effect is consistent but modest—a few percentage points—which is the honest magnitude a lightweight reliability estimator buys here.

F. Summary of findings

(i) A deadline- and fidelity-aware satisficing orchestrator substantially outperforms fidelity- and latency-greedy scheduling under contention, widening to $\sim 2\times$ at overload and near-parity when uncontended. (ii) Least-laxity ordering is the dominant contributor, sustaining $>99\%$ deadline-hit rate. (iii) The operating point is robust to the fidelity-weighting knob. (iv) Online adaptation is negligible under stationarity but recovers a consistent fraction of goodput after device degradation.

VII. LIMITATIONS AND THREATS TO VALIDITY

We are explicit about what this study does and does not establish. *First*, results are from simulation, not hardware; the fidelity model (1)–(3) is a first-order abstraction omitting crosstalk, correlated/non-Markovian noise, native-gate compilation, connectivity/SWAP overhead, readout error, and realistic entanglement-distribution scheduling and buffering.

Absolute numbers are relative comparisons within a common model, not device predictions. *Second*, the workload regime was chosen so placement is consequential (a ≈ 0.70 feasibility ceiling); where nearly all tasks are trivially feasible or infeasible, orchestration matters less and gaps shrink. *Third*, the flat weight-sensitivity (Section VI-C) is specific to this regime: when most tasks have a single feasible host, the fidelity knob has little routing leverage; richer device redundancy would widen the trade-off. *Fourth*, the learner is a tabular per-state/per-device estimator; richer function approximation might extract more from non-stationarity at the cost of the interpretability we relied on. *Fifth*, we assume gang-scheduled distributed execution on simultaneously free nodes and do not model an entanglement-generation control plane; a joint circuit-cutting-plus-orchestration formulation is future work. These caveats bound the claims to: *within a transparent, reproducible model, structured multi-objective orchestration is markedly more robust than greedy baselines, its operating point is stable, and cheap online adaptation adds a small, consistent benefit under drift.*

VIII. CONCLUSION

We framed noise-aware DQC as an AI-driven, multi-objective orchestration problem and introduced AMOS, which orders by laxity, sacrifices fidelity while optimizing deadline feasibility, speed, and load, and adapts to calibration drift via an online reliability derate. In a discrete-event simulator of a heterogeneous NISQ fleet with entanglement-mediated distribution, AMOS matched the best baseline when uncontended and improved goodput by up to $\sim 2\times$ under overload, driven mainly by least-laxity ordering and satisficing placement; its operating point was robust to the fidelity-weighting knob, and online adaptation helped modestly and only under non-stationarity. Future work includes hardware-calibrated noise models, regimes with greater routing redundancy that expose a wider multi-objective front, a joint circuit-cutting/orchestration formulation, an explicit entanglement-distribution control plane, and richer learned reliability models evaluated against the interpretable baseline established here.

REFERENCES

- [1] M. Caleffi, M. Amoretti, D. Ferrari, J. Illiano, A. Manzalini, and A. S. Cacciapuoti, "Distributed quantum computing: A survey," *Computer Networks*, vol. 254, p. 110672, 2024.
- [2] D. Barral *et al.*, "Review of distributed quantum computing: From single QPU to high performance quantum computing," *Computer Science Review*, vol. 57, p. 100747, 2025.
- [3] M. Chehimi and W. Saad, "Physics-informed quantum communication networks: A vision toward the quantum internet," *IEEE Network*, vol. 36, no. 5, pp. 32–38, 2022.
- [4] C. Piveteau and D. Sutter, "Circuit knitting with classical communication," *IEEE Trans. Inf. Theory*, vol. 70, no. 4, pp. 2734–2745, 2024.
- [5] Z. Du, W. Zhang, W. Wei, J. Chen, T. Han, Z. Liang, and Y. Mao, "Efficient circuit cutting and scheduling in a multi-node quantum system with dynamic EPR pairs," *arXiv preprint arXiv:2412.18709*, 2024.
- [6] A. Carrera Vazquez, C. Tornow, D. Ristè, S. Woerner, M. Takita, and D. J. Egger, "Combining quantum processors with real-time classical communication," *Nature*, vol. 636, p. 75, 2024.
- [7] S. Bahrani, R. D. Oliveira, J. M. Parra-Ullauri, R. Wang, and D. Simeonidou, "Resource management and circuit scheduling for distributed quantum computing interconnect networks," *arXiv preprint arXiv:2409.12675*, 2024.
- [8] H. T. Nguyen, M. Usman, and R. Buyya, "DRLQ: A deep reinforcement learning-based task placement for quantum cloud computing," in *Proc. 2024 IEEE 17th Int. Conf. on Cloud Computing (CLOUD)*, 2024, pp. 475–481.
- [9] H. T. Nguyen, M. Usman, and R. Buyya, "QFOR: A fidelity-aware orchestrator for quantum computing environments using deep reinforcement learning," *ACM Trans. Quantum Computing*, 2025.
- [10] W. Luo, J. Zhao, T. Zhan, and Q. Guan, "Adaptive job scheduling in quantum clouds using reinforcement learning," *arXiv preprint arXiv:2506.10889*, 2025.
- [11] N. K. Chandra *et al.*, "Network operations scheduling for distributed quantum computing," *arXiv preprint arXiv:2511.13687*, 2025.
- [12] D. Main, P. Drmota, D. P. Nadlinger, E. M. Ainley, A. Agrawal, B. C. Nichol, R. Srinivas, G. Araneda, and D. M. Lucas, "Distributed quantum computing across an optical network link," *Nature*, vol. 638, no. 8050, pp. 383–388, 2025.
- [13] H. T. Larasati and B.-S. Choi, "Towards fault-tolerant distributed quantum computation (FT-DQC): Taxonomy, recent progress, and challenges," *ICT Express*, vol. 11, no. 3, pp. 417–435, 2025.
- [14] J. Knörzer, X. Liu, B. F. Schiffer, and J. Tura, "Distributed quantum information processing: A review of recent progress," *arXiv preprint arXiv:2510.15630*, 2025.
- [15] A. Meddeb, "Quantum internet building blocks: State of research and development," *Computer Networks*, vol. 261, p. 111151, 2025.
- [16] H. Harada, K. Wada, and N. Yamamoto, "Doubly optimal parallel wire cutting without ancilla qubits," *PRX Quantum*, vol. 5, no. 4, p. 040308, 2024.
- [17] L. Schmitt, C. Piveteau, and D. Sutter, "Cutting circuits with multiple two-qubit unitaries," *Quantum*, vol. 9, p. 1634, 2025.
- [18] A. W. Harrow and A. Lowe, "Optimal quantum circuit cuts with application to clustered Hamiltonian simulation," *PRX Quantum*, vol. 6, p. 010316, 2025.
- [19] D. Ferrari, S. Carretta, and M. Amoretti, "A modular quantum compilation framework for distributed quantum computing," *IEEE Trans. Quantum Eng.*, vol. 4, pp. 1–13, 2023.
- [20] P. Escofet, A. Ovide, C. G. Almudéver, E. Alarcón, and S. Abadal, "Hungarian qubit assignment for optimized mapping of quantum circuits on multi-core architectures," *IEEE Comput. Archit. Lett.*, vol. 22, no. 2, pp. 161–164, 2023.
- [21] A. Pastor, P. Escofet, S. B. Rached, E. Alarcón, P. Barlet-Ros, and S. Abadal, "Circuit partitioning for multi-core quantum architectures with deep reinforcement learning," *arXiv preprint arXiv:2401.17976*, 2024.
- [22] D. Ferrari, M. Bandini, and M. Amoretti, "Execution management of distributed quantum computing jobs," in *Proc. IEEE Int. Conf. Quantum Computing and Engineering (QCE)*, 2024, pp. 150–154.
- [23] H. T. Nguyen, P. Krishnan, D. Krishnaswamy, M. Usman, and R. Buyya, "Quantum cloud computing: A review, open problems, and future directions," *arXiv preprint arXiv:2404.11420*, 2024.
- [24] H. T. Nguyen, M. Usman, and R. Buyya, "iQuantum: A toolkit for modeling and simulation of quantum computing environments," *Softw. Pract. Exper.*, vol. 54, no. 6, pp. 1141–1171, 2024.